

LAPORAN MODUL 6 PRAKTIKUM PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 6

ENKAPSULASI DAN KOMPOSISI

A. TUJUAN PRAKTIKUM

Dapat mengimplementasi konsep encapsulasi pada aplikasi serta dapat membuat aplikasi yang berelasi komposisi

B. DASAR TEORI

Enkapsulasi

Adalah salah satu konsep fundamental dalam *object oriented*, lainnya adalah pewarisan, polimorfisme dan abstrak. Enkapsulasi adalah suatu cara pengemasan data dan fungsi dalam sebuah kelas yang terlindungi dari akses secara sembarangan dari pihak luar. Oleh karena itu disebut sebagai proses pengkapsulan, dimana seperti obat bubuk yang terlindungi oleh bungkus kapsul supaya tidak mudah terkontaminasi bakteri. Teknik melakukan enkapsulasi adalah dengan memberikan hak akses private untuk atribut kelas, lalu untuk mengaksesnya dilakukan melalui method yang diberi hak akses public. Jika atribut dideklarasikan dengan hak akses private maka tidak dapat diakses dari luar kelas, maka data tersebut terlindungi. Keuntungan menggunakan enkapsulasi adalah

dapat memodifikasi suatu implementasi program tanpa perlu membongkar kode aslinya.

Komposisi

Dalam *object oriented*, satu kelas dapat berelasi dengan kelas lain untuk menggunakan fungsionalitas *method* yang disediakan oleh kelas lain. Hubungan relasi antar kelas diantaranya adalah komposisi. Dua buah kelas dikatakan memiliki relasi komposisi jika salah satu kelas bersifat mempunyai(*own*) kelas yang lain dan apabila pemilik kelas (*owner*) dihapuskan maka kelas yang lain tidak bisa berfungsi. Contohnya kelas manusia mempunyai kelas kepala, kelas tangan, kelas kaki. Bila kelas manusia tersebut dihapus maka kelas kepala, tangan, kaki tidak bisa berfungsi.

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1

```
public class Pegawai {  
    private String nama;  
    private String jabatan;  
    private int gaji;  
  
    public String getJabatan() {  
        return jabatan;  
    }  
  
    public void setJabatan(String jabatan) {  
        this.jabatan = jabatan;  
    }  
  
    public long getGaji() {  
        return gaji;  
    }  
  
    public void setGaji(int gaji) {  
        this.gaji = gaji;  
    }  
  
    public String getNama() {  
        return nama;  
    }  
  
    public void setNama(String nama) {  
        this.nama = nama;  
    }  
  
    public void cetakPegawai() {  
        System.out.println("");  
        System.out.println("Nama Pegawai:" + this.nama);  
        System.out.println("Jabatan:" + this.jabatan);  
        System.out.println("Gaji Pokok \t:" + this.gaji);  
    }  
}
```

Pembahasan : Program diatas Adalah contoh enkapsulasi kelas pegawai punya 3 atribut dengna getter dan setter nya , kemudian void cetakPegawai untuk menampilkan apa yang ada di kelas ini jika dipanggil oleh kelas Main

2. Praktik 2

```
public class TestEnkapsulasi {  
    public static void main(String[] args) {  
        Pegawai dataPeg = new Pegawai();  
        dataPeg.setNama("Budi");  
        dataPeg.setJabatan("Supervisor");  
        dataPeg.setGaji(4000000);  
        dataPeg.cetakPegawai();  
    }  
}
```

```
ktikum Paradigma Pemrograman\6'; & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\fb39a0ed2a114fcc0ff93bed16068cef\redhat.java\jdt_ws\6_a0585e62\bin' 'TestEnkapsulasi'
```

```
Nama Pegawai:Budi  
Jabatan:Supervisor  
Gaji Pokok      :4000000  
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\6>
```

Pembahasan : program kedua untuk memanggil enkapsulasi pada program pertama dengan isi main , dibuat obyek baru dataPeg, di isikan value dengan setter masing masing atribut lalu fungsi cetakPegawai di panggil untuk menampilkan value yang diberikan di program main

3. Praktik 3

```
public class Perusahaan {
    private String nmPerush;
    private String alamat;
    private Pegawai peg; //komposisi dengan relasi has-a

    public Perusahaan() {
        this.nmPerush = "PT. Lancar Jaya";
        this.alamat = "Yogyakarta";
        this.peg = new Pegawai();
        peg.setNama("David");
        peg.setJabatan("Manager");
        peg.setGaji(5000000);
    }

    public void cetakPerush() {
        System.out.println("Nama Perusahaan:" + this.
nmPerush);
        System.out.println("Alamat : " + this.alamat);
        peg.cetakPegawai();
    }
}
```

Pembahasan : dalam program ini `Perusahaan` memiliki sebuah objek dari kelas `Pegawai`. Hal ini terlihat jelas pada deklarasi atribut di dalam kelas `Perusahaan`, yaitu `private Pegawai peg;`. Atribut `peg` ini adalah sebuah instance dari kelas `Pegawai`, yang menunjukkan bahwa setiap objek `Perusahaan` akan memiliki satu objek `Pegawai` di dalamnya. Di dalam constructor kelas `Perusahaan`, yaitu `public Perusahaan()`, dilakukan inisialisasi untuk atribut-atribut milik `Perusahaan` itu sendiri, yaitu `nmPerush` dan `alamat`. Selain itu, di dalam constructor ini juga dibuat sebuah objek baru dari kelas `Pegawai` melalui `this.peg = new Pegawai();`. Setelah objek `peg` dibuat, nilai-nilai atribut dari objek `peg` tersebut diatur menggunakan method `setter` dari kelas `Pegawai`. Kelas `Perusahaan` juga memiliki sebuah method bernama `cetakPerush()` yang berfungsi untuk menampilkan informasi perusahaan dan juga informasi pegawai yang dimilikinya. Method `cetakPerush()` ini memanggil `peg.cetakPegawai()` untuk menampilkan detail dari objek `Pegawai` yang menjadi bagian dari `Perusahaan`.

4. Praktik 4

```
public class
TestKomposisi {
    public static void
main(String[] args) {
        Perusahaan
kantor1 = new Perusahaan
();
        kantor1.
cetakPerush();
    }
}
```

```
Ctrl+Messages - cp C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\F039a0ed2a114fcc0ff93bed160e8cef\redhat_java\jdk_ws\6_a0985ee2\bin TestKomposisi
Nama Perusahaan:PT. Lancar Jaya
Alamat :Yogyakarta

Nama Pegawai:David
Jabatan:Manager
Gaji Pokok :5000000
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\6>
```

Pembahasan : Kelas `TestKomposisi` digunakan untuk menguji kelas `Perusahaan`. Di dalam method `main` pada kelas `TestKomposisi`, sebuah objek dari kelas `Perusahaan` dibuat dengan nama `kantor1`. Kemudian, method `kantor1.cetakPerush()` dipanggil untuk menampilkan semua informasi yang terkait dengan objek `kantor1`, termasuk data pegawai yang ada di dalamnya.

LATIHAN

1. Latihan 1

```
public class SepedaMotor {
    private String merk;
    private int cc;
    private String warna;
    private int tahun;
    private double harga;

    public String getMerk() {
        return merk;
    }

    public void setMerk(String merk) {
        this.merk = merk;
    }

    public int getCc() {
        return cc;
    }

    public void setCc(int cc) {
        this.cc = cc;
    }

    public String getWarna() {
        return warna;
    }

    public void setWarna(String warna) {
        this.warna = warna;
    }

    public int getTahun() {
        return tahun;
    }

    public void setTahun(int tahun) {
        this.tahun = tahun;
    }

    public double getHarga() {
        return harga;
    }

    public void setHarga(double harga) {
        this.harga = harga;
    }

    public void tampilkanInfo() {
        System.out.println("Merk: " + this.merk);
        System.out.println("CC: " + this.cc);
        System.out.println("Warna: " + this.warna);
        System.out.println("Tahun: " + this.tahun);
        System.out.println("Harga: " + this.harga);
    }
}
```

```

import java.util.Scanner;

public class TestSepedaMotor {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        SepedaMotor motor = new SepedaMotor();

        System.out.print("Masukkan Merk: ");
        motor.setMerk(input.nextLine());

        System.out.print("Masukkan CC: ");
        motor.setCc(input.nextInt());
        input.nextLine(); // Consume newline

        System.out.print("Masukkan Warna: ");
        motor.setWarna(input.nextLine());

        System.out.print("Masukkan Tahun: ");
        motor.setTahun(input.nextInt());

        System.out.print("Masukkan Harga: ");
        motor.setHarga(input.nextDouble());

        System.out.println("\nInformasi Sepeda Motor:");
        motor.tampilkanInfo();

        input.close();
    }
}

```

```

PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\6> d:; cd "d:\AREAS\Semester3\Praktikum Paradigma Pemrograman\6"; & 'C:\Program Files\Java\jdk-23\bin\java
.exe' "-XX:+ShowCodeDetailsInExceptionMessages" "-cp" "C:\Users\ASUS\AppData\Roaming\code\User\workspaceStorage\fb39a0ed2a114fcc0ff93bed16068cef\redhat.java\jdt_
ws\6_a0585e62\bin" "TestSepedaMotor"
Masukkan Merk: susuki
Masukkan CC: 120
Masukkan Warna: pink
Masukkan Tahun: 2003
Masukkan Harga: 17000000

Informasi Sepeda Motor:
Merk: susuki
CC: 120
Warna: pink
Tahun: 2003
Harga: Rp17,000,000
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\6>

```


Pembahasan : Pada kelas `SepedaMotor`, terdapat lima buah atribut yaitu `merk`, `cc`, `warna`, `tahun`, dan `harga`. Semua atribut ini, yaitu `merk`, `cc`, `warna`, `tahun`, dan `harga`, dideklarasikan dengan hak akses `private` untuk menerapkan prinsip enkapsulasi. Dengan hak akses `private`, atribut-atribut tersebut tidak bisa diakses secara langsung dari luar kelas `SepedaMotor`. Untuk berinteraksi dengan atribut-atribut `private` ini, disediakan serangkaian method `public`. Terdapat method `setter` seperti `setMerk()`, `setCc()`, `setWarna()`, `setTahun()`, dan `setHarga()` yang digunakan untuk mengisi atau mengubah nilai dari masing-masing atribut. Ada juga method `getter` seperti `getMerk()`, `getCc()`, `getWarna()`, `getTahun()`, dan `getHarga()` yang berfungsi untuk mengambil nilai dari atribut-atribut tersebut.

Kelas `TestSepedaMotor` program dibuat menjadi dinamis dengan menerima input dari pengguna menggunakan kelas `Scanner`. Pertama, sebuah objek `Scanner` bernama `input` dan sebuah objek `SepedaMotor` bernama `motor` dibuat. Program kemudian meminta pengguna untuk memasukkan data untuk setiap atribut `SepedaMotor` satu per satu. Setiap input dari pengguna disimpan ke dalam atribut objek `motor` dengan memanggil method `setter` yang sesuai, misalnya `motor.setMerk(input.nextLine())` untuk mengatur merk. Setelah semua data dimasukkan oleh pengguna, program memanggil method `motor.tampilkanInfo()` untuk mencetak semua informasi sepeda motor yang telah dimasukkan ke layar.

D. PEMBAHASAN TUGAS

E. KESIMPULAN

dapat disimpulkan bahwa enkapsulasi adalah salah satu pilar penting dalam pemrograman berorientasi objek yang bertujuan untuk membungkus data (atribut) dan method yang mengoperasikannya ke dalam satu unit tunggal yang disebut kelas. Dengan menggunakan hak akses `private` pada atribut-atribut seperti `nama` dan `gaji` pada kelas `Pegawai`, atau `judul` dan `harga` pada kelas `Buku`, kita dapat melindungi data dari akses dan modifikasi yang tidak sah dari luar kelas. Akses terhadap data tersebut diatur secara terkendali melalui method publik, yaitu `setter` untuk mengubah nilai dan `getter` untuk membaca nilai. Selain itu, praktikum ini juga memperkenalkan konsep komposisi, yang merupakan bentuk relasi "memiliki sebuah" (has-a) antar kelas. Seperti yang ditunjukkan pada kelas `Perusahaan` yang memiliki objek dari kelas `Pegawai` sebagai atributnya (`private Pegawai peg;`), komposisi memungkinkan kita untuk membangun kelas yang lebih kompleks dengan menggabungkan objek-objek dari kelas lain yang lebih sederhana.

F. LAMPIRAN

